

→ Grammatik der Progsprache
verletzt

ationsregel für zusammengefügte Zeichen

operatoren

bool < char < int < unsigned int < float < double

bool < char < int < unsigned int < float < double

Expr ++	Wert von expr um 1 erhöht, <u>alte</u> Wert wird als R-Wert zurückgegeben
++ Expr	Wert von expr um 1 erhöht, <u>neue</u> Wert wird als L-Wert zurückgegeben
Expr --	Wert von expr um 1 erniedrigt, <u>alte</u> Wert wird als R-Wert zurückgegeben
-- Expr	Wert von expr um 1 erniedrigt, <u>neue</u> Wert wird als L-Wert zurückgegeben

$$F^*(\beta, p, e_{\min}, e_{\max})$$

p ist immer
Anzahl Ziffern
allgemein

$$\mathbb{F}^*(\beta, \rho, e_{\text{min}}, e_{\text{max}})$$
$$\pm d_1 \cdot d_2 \cdot \dots \cdot d_n \cdot \beta^0 \quad d_i \neq 0$$

→ + Negative

Anzahl: $2 \cdot (b-1) \cdot b^{p-1} \cdot (e_{\max} - e_{\min} + 1)$

+ 8 Bits. Wertebereich umfasst die 2^8 ganzen Zahlen: Der Typ unsigned int

$$\{-2^{B-1}, -2^{B-1}+1, \dots, -1, 0, 1, \dots, 2^{B-1}-2, 2^{B-1}-1\}$$

- Wertebereich : $\{0, 1, \dots, 2^8 - 1\}$

Vektoren `std::vector<int> vec;`

`vec.push_back(...);` `vec.pop_back();`

`string s = "abc";`
`vec.push_back(s);` → `vec[0]` → `s`
 ↑
 `vec.at(i);`

ist konstante Ganzzahl

- Sei c ein Container
 - Zeiger vorrücken:
 - Iterator auf i^{tes} Element: $it=c.begin()$
 - Iterator hinter letztes Element: $it=c.end()$

ist Zeiger auf konst Ganzzahl

ist konstanter Zeiger auf Ganzzahl

ist konstanter Zeiger auf konst Ganzzahl

```
it: std::vector::const_iterator it
```

switch	Fallunterscheidung
Wird ein case mit einem break abgeschlossen, so werden die darunter liegenden cases auch noch ausgeführt, bis ein break erreicht wird. Die einzelnen Untersuchungswerte müssen Konstanten sein.	
<pre>std::cout << "Behind which door (1,2,3) is the prize?"; int door_number; std::cin >> door_number; switch (door_number) { case 1: break; case 3: std::cout << "Wrong choice :-(\n"; break; case 2: std::cout << "You won the prize!\n"; break; default: std::cout << "Error: unknown door number.\n"; }</pre> // User inputs 0 --> Error: unknown door number. // User inputs 1 --> Wrong choice :-(// User inputs 2 --> You won the prize! // User inputs 3 --> Wrong choice :-(	

Vektoren (mehrdim.)	mehrdimensionale "Massenvariable" eines bestimmten Typs
Erforderd:	<code>#include<vector></code>
Wichtige Befehle:	Definition: <code>std::vector<T>: vector<T> ></code> <code>my_vec (n,zero, std::vector<T>(n,0,la,init,value))</code> Zugriff: <code>my_arr.at(1).at(1) = 8</code> <code>my_arr.at(0).at(2);</code> (Anstatt last gehen natürlich auch andere Typen).
Beispiel:	<pre>std::vector<std::vector<int>>> my_vec (2, std::vector<int>(4, 0)); my_vec.at(1).at(2) = 3; // my_vec becomes // 0, 0, 0, 0 // 0, 0, 3, 0</pre>

<code>std::string</code>	komfortablerer Datentyp für Zeichen
--------------------------	-------------------------------------

Erfordert: `#include<string>`

```
variable Länge:      std::string my_str (n, 'a');
                     (n kann variabel sein)
Länge abfragen:      my_str.length()
vergleichbar:         text1 == text2
hintereinander hängen: text1 += text2
bequemer Output:     std::cout << my_str;
```

Rekursion	Selbstaufruf einer Funktion
-----------	-----------------------------

Jeder rekursive Funktionsaufruf hat seine eigenen, unabhängigen Variablen und Argumente. Dies kann man sich sehr gut anhand des in der Vorlesung gezeigten Stacks vorstellen (`fac` ist im Beispiel unten definiert):

```

graph TD
    n1["n = 1"] --> f1["fac(1)"]
    f1 -- 1 --> n2["n = 2"]
    n2 --> f2["fac(2)"]
    f2 -- 2 --> n3["n = 3"]
    n3 --> f3["fac(3)"]
    f3 -- 6 --> n4["n = 4"]
    n4 --> f4["fac(4)"]
    f4 -- 24 --> cout["std::cout << fac(4)"]
  
```

```
// POST: return value is n!
unsigned int fac (const unsigned int n)
{
    if (n <= 1) return 1;
    return n * fac(n-1); // n > 1
}
```

<code>struct</code>	Container für Datentypen
---------------------	--------------------------

Wichtige Befehle:

```

Definition:      struct str_name {
                  int mem1;
                  bool mem2;
                  int mem3;
                };

Objekt erstellen:
mit Startwerten:  str_name obj1;
                  str_name obj2 = {3, true, 4};
aus anderem Objekt: str_name obj3 = obj2;
Zugriff auf Member: obj1.mem1

```

Die *Definition* eines Structs hat ein ; am Schluss

Nur der Zuweisungsoperator (=) wird automatisch erstellt (und kopiert dann die Member einzeln). Die anderen Operatoren (z.B. ==, !=, ...) muss man selbst passend überladen (siehe Eintrag `operator...`).

Bei der **Default-Initialisierung** eines Objekts des Typs `str_name` werden alle Member einzeln default-initialisiert. Für fundamentale Typen (`int`, `float`, usw.) bedeutet das, dass sie *uninitialisiert* sind, bis man ihnen nachträglich einen Wert zuweist. Das führt zu Problemen, falls man ihren Wert vorher schon ausliest.

```
struct candidate {
    std::string name;    // Name of the participant
    int age;             // Her/his age
};
```

```
int main () {
    // Initialization
    candidate mary;           // default-initialisation
    std::cout << mary.age;    // Undefined behavior
    mary.name = "Mary"; mary.age = 43;
    std::cout << mary.age;    // Problem gone: mary.age is 43
    candidate bob = {"Bob", 28}; // using starting values
    candidate fred = bob;      // using other object
    return 0;
}
```

Klassen ~ Theorie

class	Datencontainer mit Kapselung	Memberfunktion	Funktionalität auf Klassen
Eine Klasse besteht aus Daten und Funktionen, genannt Member, und erlaubt deren Kapselung via Zugriffskontrolle. Auf Member im privaten Teil (<i>private</i>) einer Klasse kann nur durch die Klasse selbst, d.h., deren Member-Funktionen zugegriffen werden.		Memberfunktionen stellen Funktionalität auf einer Klasse bereit. Sie ermöglichen den kontrollierten Zugang zu den privaten Daten und privaten Memberfunktionen. Die <i>Deklaration</i> einer Memberfunktion erfolgt immer in der Klassendefinition, die <i>Definition</i> der Memberfunktion ist auch extern möglich (ermöglicht vorkompilierte Libraries). Dann muss allerdings die Zugehörigkeit zur Klasse explizit erwähnt werden mittels der <i>::Schreibweise</i> .	Der Aufruf einer Memberfunktion ist <code>obj.mem_func(arg1, arg2, ..., argN)</code> . Der Teil <code>obj.</code> kann weggelassen werden, falls aus der Class heraus auf einen Member des aufrufenden Objekts (siehe Eintrag <i>*this</i>) zugegriffen wird.
Zugriff von außerhalb der Klasse muss über öffentliche (<i>public</i>) Member erfolgen. Per default sind die Member einer Klasse privat.		<i>// Internal Definition vs. External Definition</i> <pre>class Insurance { public: void set_rate_l(const double v) { rate = v; } // int. void set_rate_e(const double v); ... private: double rate; ... }; void Insurance::set_rate_e(const double v) {rate = v; } // ext.</pre>	
<pre>class my_class { public: // public section double some_public_member; private: // private section double some_private_member; }; ... my_class inst; inst.some_public_member = 1.0; inst.some_private_member = 0.0; // ERROR: cannot access private // members directly</pre>			

const Memberfunktion	Unverändernde Memberfunktion	Konstruktor	Datencontainer Initialisierung
Das <i>const</i> bezieht sich auf <i>*this</i> . Es verspricht, dass durch die Funktionsausführung das implizite Argument nicht im Wert verändert wird.		Konstrukturen sind spezielle Memberfunktionen einer Klasse, die den Namen der Klasse tragen. Sie werden bei der Variablen Deklaration aufgerufen.	
<pre>class Insurance { public: double get_value() const { return value; // same: return (*this).value; } ... // e.g. members which set the data members private: double value; };</pre>		Sie werden analog zu Funktionen überladen und bei der Variablen Deklaration wie eine Funktion aufgerufen. Damit das funktioniert, muss der Konstruktor öffentlich (<i>public</i>) sein.	
		Spezielle Konstrukturen sind der <i>Default-Konstruktor</i> (kein Argument), welcher automatisch erzeugt wird, falls eine Klasse keinen Konstruktor definiert, und der <i>Konversions-Konstruktor</i> (genau ein Argument), welcher die Definition <i>benutzerdefinierter Konversionen</i> ermöglicht.	
<pre>operator... // Einem Operator überladen.</pre>		<i>*this</i>	Zugriff auf implizites Argument
Operator-Überladung wird zum Beispiel verwendet, um Operatoren (+, -, *, etc.) auf eigenen Structs zu definieren.		Memberfunktionen einer Klasse haben ein implizites Argument, nämlich das aufrufende Objekt. Und <i>this</i> ist ein Zeiger darauf. Via <i>*this</i> kann man darauf zugreifen.	
Mittels dem <i>operator...</i> Keyword ist es ebenfalls möglich, den Operator auszuführen. Das sollte man aber vermeiden, da damit der Code unlesbar wird.		Bei Zugriffen von innerhalb einer Klasse aus auf Daten-Member oder Member-Funktionen wird das implizite Argument automatisch verwendet. Man muss es dann also <i>nicht explizit angeben</i> (siehe Eintrag <i>Memberfunktion</i>). Man muss <i>*this</i> aber mindestens explizit verwenden, falls z.B. eine <i>Referenz</i> auf das implizite Argument zurückgegeben werden soll.	
<pre>struct rational { int n; int d; // INV: d != 0 }; // POST: return value is the sum of a and b rational operator+ (const rational a, const rational b) { rational result; result.n = a.n * b.d + a.d * b.n; result.d = a.d * b.d; return result; } // POST: return value is the sum of a and b rational operator* (const rational a, const int b) { rational b_rat; b_rat.n = b; b_rat.d = 1; // b_rat is b/1 return a * b_rat; // Use operator+ for two rationals (above) } int main () { rational r = {1, 2}; rational s = {3, 4}; rational t = r + s; // first overload std::cout << t.n << "/" << t.d << "\n"; // Output: 10/8 rational u = r * 3; // second overload std::cout << u.n << "/" << u.d << "\n"; // Output: 7/2 return 0; }</pre>		<pre>// General example class Human { public: void set(const int a) { age = a; } // or (*this).age = a; void print1() const { std::cout << (*this).age; } void print2() const { std::cout << age; } // equivalent private: int age; }; Human me; me.set(175); me.print1(); // 175 me.print2(); // 175 // Another example class Complex { public: // Note: In most applications // a reference should be returned. Complex operator+ (const Complex & b) { real += b.real; imag += b.imag; return *this; } ... // other members private: float real; float imag; };</pre>	

Copy-Konstruktor	Kopier-Initialisierung
Der <i>Copy-Konstruktor</i> ist der Konstruktor, dessen Argumenttyp <i>const My_Class</i> ist.	
<pre>struct Customer { std::string name; int duration; int amount_insured; }; class Insurance { public: Insurance (const Insurance& rhs) : length (rhs.length), ... // copy remaining data mbrs { cust = new Customer (length); for (int i = 0; i < length; ++i) cust[i] = rhs.cust[i]; } ... // other public members private: Customer* cust; // pointer to an array containing customers int length; // length of cust ... // other private members };</pre>	

operator=	Kopier-Zuweisung
Eng verwandt mit <i>operator=</i> ist der <i>Copy-Konstruktor</i> . Der Unterschied ist, dass der <i>Copy-Konstruktor</i> nur bei der <i>Initialisierung</i> aufgerufen wird, <i>operator=</i> hingegen nur <i>nach</i> der Initialisierung. z.B.	
<pre>my_class a (5, 6), c (4, 4); // Call a general constructor my_class b = a; // Call copy-constructor c = b; // Call operator=</pre>	
<i>operator=</i> kann anders als der <i>Copy-Konstruktor</i> implementiert werden müssen. Ein Beispiel sind Klassen, welche Pointer auf dynamisch generierte Objekte als Member haben. Dann muss bei <i>operator=</i> meistens zuerst das aktuell vorhandene Objekt gelöscht werden, bevor die Kopie erstellt werden kann. Dies ist beispielsweise beim Stack aus der Vorlesung relevant.	
<i>operator=</i> gibt im Normalfall eine <i>Referenz</i> auf seinen linken Operanden zurück.	
Faustregel: Meistens führt <i>operator=</i> zuerst die Aufgaben des <i>Destructors</i> , und dann die Aufgaben des <i>Copy-Konstruktors</i> aus.	

<i>// Call from Inside vs. Call from Outside</i> <pre>class Insurance { public: double get_rate () { if (!is_up_to_date) update_rate(); // from inside return rate; } double get_cost () {return get_rate() * ...; } // from inside ... // e.g. stuff which sets the data members private: bool is_up_to_date; double rate; double update_rate () { rate = ...; } }; ... Insurance insurance; ... std::cout << insurance.get_rate(); // from outside</pre>	
--	--

<pre>class Insurance { public: Insurance(double v, int r) // general constructor : value (v), rate (r) // initialize data members { update_rate(); } Insurance() // default constructor : value (0), rate (0) // initialize data members { } // other members private: double value; double rate; void update_rate(); }; ... // General Constructor Insurance i1 (10000, 10); // default-Constructor, direct call Insurance i3; // identical: Insurance i3 (); ... class Complex { public: // Conversion Constructor (float --> Complex) Complex(const float m1) : real (i1), imag (0) { } private: float real; float imag; };</pre>	
---	--

<pre>...->...</pre>	Auf einen Member eines Objekts zugreifen, auf das ein Pointer gegeben ist.
<i>ptr->mem</i> macht das Selbe wie <i>(*ptr).mem</i> .	
<pre>struct my_class { // POST: "Hi!" is written to std::cout void say_hi () const { std::cout << "Hi! "; } }; ... my_class obj; my_class* ptr = &obj; // just a pointer to obj obj.say_hi(); // direct access ptr->say_hi(); // using -> (*ptr).say_hi(); // equivalent</pre>	

Destructor	Class abbauen
<i>// for Customer-struct see example on Copy-Constructor</i> <pre>class Insurance { public: Insurance& operator= (const Insurance& rhs) { // Cleanup of current customers delete[] cust; // Copy over the customers from rhs cust = new Customer (length); for (int i = 0; i < length; ++i) cust[i] = rhs.cust[i]; length = rhs.length; ... // copy other data members return *this; // return a reference to left operand } ... // other members }; ... // for Customer-struct see example on Copy-Constructor class Insurance { public: ~Insurance () { delete[] cust; } // free dynamic space ... };</pre>	

new	Objekt mit dynamischer Lebensdauer erstellen.
Mit new wird ein Objekt erstellt, indem der nötige Speicherplatz reserviert wird, und dann ein gegebener Konstruktor aufgerufen wird.	
Der Rückgabewert von new ist ein Pointer auf das neu erstellte Objekt.	
<pre> Class MyClass { public: MyClass (const int i) : y (i) { std::cout << "Hallo"; } int get_y () { return y; } private: int y; }; ... MyClass* ptr = new MyClass (3); // outputs Hello MyClass* ptr2 = ptr; // another pointer to the new object std::cout << (*ptr).get_y(); // Output: 3 ... </pre>	
new ... []	Ranges mit dynamischer Lebensdauer und Länge erstellen.
<pre> int n; std::cin >> n; int* range = new int[n]; // Read in values to the range for (int* i = range; i < range + n; ++i) std::cin >> *i; </pre>	

Iterator (auf Vektor)	Iterieren über einen Vektor.
Im Folgenden wird nur auf die Unterschiede zum Zeiger (auf Array) eingegangen. Die restliche Bedienung erfolgt gleich.	
Erfordert: <code>#include <vector></code>	
Wichtige Befehle (gelte <code>std::vector<int> a (6, 0);</code>):	
Definition: <code>std::vector<int>::iterator itr = ...;</code> Iterator auf a.at(0): <code>a.begin()</code> Past-the-End-Iterator: <code>a.end()</code>	
Anstelle des ... in der Definition eines Iterators müssen andere Iteratoren stehen (z.B. <code>a.begin()</code>).	
<pre> // Example for vectors. // To avoid the lengthy lines see entry on typedef. // Read 6 values into a vector std::cout << "Enter 6 numbers:\n"; std::vector<int> a (6, 0); for (std::vector<int>::iterator i = a.begin(); i < a.end(); ++i) std::cin >> *i; // read into object of iterator // Output: a.at(0)=a.at(3), a.at(1)=a.at(4), a.at(2)=a.at(5) for (std::vector<int>::iterator i = a.begin(); i < a.end(); ++i) { assert(i+3 < a.end()); // Assert that i+3 stays inside. std::cout << (*i + *(i+3)) << ", "; } </pre>	

const (Iterator)	kein Schreibzugriff auf das Objekt
Vorsicht: Einen const -Iterator erzeugt man mittels <code>std::vector<int>::const_iterator ...</code> und nicht mittels <code>const std::vector<int>::iterator ...</code>	
Die zweite Variante erzeugt einen Iterator, den man nicht herumschieben kann. In dieser Vorlesung gehen wir aber nur auf die Iteratoren näher ein, welche den Schreibzugriff auf das Objekt verbieten (erste Variante oben).	
<pre> std::vector<int> a (6, -8); // a is: -8 -8 -8 -8 -8 -8 std::vector<int>::const_iterator itr = a.begin() + 3; *itr = 4; // NOT valid itr = a.begin(); // valid (itr now points to a.at(0)) </pre>	
Bereichsbasierte for-Schleife	
Sequenzielle Iteration mittels eines Iterators über einen llvec (const -Iteratormöglich; andere Container möglich): <pre> llvec v(3); // v == 0, 0, 0 for (llvec::iterator it = v.begin(); it != v.end(); ++it) { std::cout << *it; // 000 } </pre> Kann alternativ auch wie folgt geschrieben werden: <pre> for (int i : v) std::cout << i; // 000 </pre> Wird dann zu Iterator-basierter Schleife übersetzt.	
Modifizierender Zugriff ist auch möglich: <pre> for (int& i : v) i += 3; for (int i : v) std::cout << i; // 369 </pre>	

Code - Examples

```

Array-based Vector, Rule of Three - Master Solution
#include <iostream>
#include <vector>
#include "avec.h"

avec::avec(unsigned int size): count(size) {
    elements = new tracked[size];
}

unsigned int avec::size() const {
    return count;
}

int& avec::operator[](unsigned int i) {
    return *elements[i];
}

int& avec::at(unsigned int i) {
    assert(i < count);
    return *elements[i];
}

void avec::print(std::ostream& sink) const {
    tracked end = elements + count;
    for (tracked p = elements; p != end; ++p) {
        sink << *(p) << " ";
    }
    sink << "\n";
}

std::ostream& operator<<(std::ostream& sink, const avec& vec) {
    vec.print(sink);
    return sink;
}

```

```

1 #include "avec.h"
2 #include <unordered_set>
3
4 avec::avec(const avec& vec) : count(vec.count) {
5     elements = new tracked[count];
6     for (unsigned int i = 0; i != count; ++i) {
7         elements[i] = vec.elements[i];
8     }
9 }
10
11 avec& avec::operator=(const avec& t) {
12     if (elements != t.elements) {
13         avec copy(t);
14         std::swap(copy.elements, elements);
15         std::swap(copy.count, count);
16     }
17     return *this;
18 }
19
20 avec::~avec() {
21     delete[] elements;
22 }
23
24 void your_own_tests() {}
25

```

Handwritten notes:
 - Line 4: Copy
 - Line 11-17: VergOp
 - Line 20-22: delete

```

Task 3: Smart Pointers - Master Solution
1 #include "smart.h"
2 #include "tracker.h"
3
4 Smart::Smart(): count(nullptr), ptr(nullptr) {}
5
6 Smart::Smart(tracked* t): count(nullptr), ptr(t) {}
7     if (t != nullptr) {
8         count = new unsigned int(1);
9     }
10
11
12 Smart::Smart(const Smart& src): count(src.count), ptr(src.ptr) {}
13     if (count != nullptr) {
14         ++count;
15     }
16
17
18 Smart::~Smart() {
19     if (count != nullptr) {
20         if (--count == 0) {
21             delete count;
22             delete ptr;
23         }
24     }
25 }
26
27 Smart& Smart::operator=(const Smart& src) {
28     if (ptr != src.ptr) {
29         Smart copy(src);
30         // std::swap swaps the content of two variables, see also
31         // https://en.cppreference.com/w/cpp/algorithm/swap
32         // This can of course also be done using a temporary local variable.
33         std::swap(count, copy.count);
34         std::swap(ptr, copy.ptr);
35     }
36     return *this;
37 }
38
39 Console

```

```

30 // std::swap swaps the content of two variables, see also
31 // https://en.cppreference.com/w/cpp/algorithm/swap
32 // This can of course also be done using a temporary local variable.
33 std::swap(count, copy.count);
34 std::swap(ptr, copy.ptr);
35 }
36 return *this;
37 }
38
39 tracked& Smart::operator*() {
40     return *ptr;
41 }
42
43 const tracked& Smart::operator*() const {
44     return *ptr;
45 }
46
47 tracked& Smart::operator->() {
48     return ptr;
49 }
50
51 const tracked& Smart::operator->() const {
52     return ptr;
53 }
54
55 bool Smart::operator==(const Smart& cmp) const {
56     return (ptr == cmp.ptr);
57 }
58
59 bool Smart::operator!=(const Smart& cmp) const {
60     return !(this == cmp);
61 }
62
63 void your_own_tests() {}
64
65 Console

```


Simply Linked List - Master Solution

```
3 // post: return the sum of all values of this list
4 unsigned int List::sum() {
5     unsigned int sum = 0;
6     for (ListNode* node = first; node != nullptr; node = node->next) {
7         sum += node->value;
8     }
9     return sum;
10 }
11
12 // pre: any value
13 // post: remove a single element with given value, if any
14 void List::deleteValue(unsigned int value) {
15     if (first == nullptr) {
16         return;
17     }
18     if (first->value == value) {
19         ListNode* node = first;
20         first = first->next;
21         delete node;
22     } else {
23         ListNode* prev = first;
24         while (prev->next->value != value && prev->next != nullptr) {
25             prev = prev->next;
26         }
27         if (prev->next != nullptr) {
28             ListNode* node = prev->next;
29             prev->next = node->next;
30             delete node;
31         }
32     }
33 }
```

Geometric Intersection - Master Solution

```
1 #include <Vec3.h>
2 #include <cmath>
3 #include <cassert>
4
5 Vec3& Vec3::operator+=(const Vec3& other) {
6     m_x += other.m_x;
7     m_y += other.m_y;
8     m_z += other.m_z;
9     return *this;
10 }
11
12 Vec3 Vec3::operator*(const Vec3& other) const {
13     Vec3 sum(m_x, m_y, m_z);
14     sum += other;
15     return sum;
16 }
17
18 Vec3& Vec3::operator=(const Vec3& other) {
19     m_x = other.m_x;
20     m_y = other.m_y;
21     m_z = other.m_z;
22     return *this;
23 }
24
25 Vec3 Vec3::operator-(const Vec3& other) const {
26     Vec3 difference(m_x, m_y, m_z);
27     difference -= other;
28     return difference;
29 }
30
31 // Console
```

EBNF: Types

EBNF - Master Solution

```
1 // Consumes next 'var'
2
3 // Consumes next 'var'
4 bool VarDec(std::istream& is) {
5     return consume(is, 'var');
6 }
7
8 // Consumes next Type
9 bool Type(std::istream& is) {
10     char ch = lookahead(is);
11     switch (ch) {
12         case 'i':
13             return consume(is, "integer");
14         case 'b':
15             return consume(is, "boolean");
16         case 'r':
17             return consume(is, "real");
18         default:
19             return false;
20     }
21 }
22
23 // Consumes next Letter
24 bool Letter(std::istream& is) {
25     char ch = peek(is);
26     if (ch == 'a' && ch != 'z') {
27         return consume(is, ch);
28     } else {
29         return false;
30     }
31 }
32
33 // Consumes next VarList
34 bool VarList(std::istream& is) {
35     if (Letter(is)) return false;
36     while (Letter(is)) {
37         return true;
38     }
39 }
40
41 // Consumes next VarList = Var { " " Var } ":" Type.
42 bool VarDecl(std::istream& is) {
43     if (Var(is)) {
44         while (consume(is, ' ')) {
45             if (!Var(is)) return false;
46         }
47         return consume(is, ':') && Type(is);
48     }
49     return false;
50 }
51
52 // Consumes next Decl = "var" VarList ":".
53 bool Decl(std::istream& is) {
54     if (VarDecl(is)) {
55         return VarList(is) && consume(is, ':');
56     }
57     return false;
58 }
59
60 // Console
```

> Console

Tree - Master Solution

```
1 // POST: Find and return the maximum leaf node value in a (sub)tree starting
2 // at root. If root is the empty tree, return 0.
3 unsigned int findMax(const Node& root) {
4     unsigned int max = 0;
5
6     // If the tree is empty, return 0
7     if (root == nullptr) {
8         return 0;
9     }
10     // If the current node is a leaf, return its value
11     if (root->isLeaf()) {
12         return root->value;
13     }
14     // Otherwise, the current node is not a leaf, and we search for the max
15     // in its children
16     for (Node* child : root->children) {
17         unsigned int childMax = findMax(child);
18         if (childMax > max) {
19             max = childMax;
20         }
21     }
22     return max;
23 }
```

Geometric Intersection - Master Solution

```
1 #include <Sphere.h>
2 #include <cmath>
3
4 bool Sphere::Intersect(const Ray& ray, float& t) const {
5     // solving the equation |x-center||^2 = radius^2
6     // with x = ray.origin + t * ray.direction
7     // with v = ray.origin - v.center the solution is
8     // t = (-v * ray.direction +/- sqrt((v * ray.direction)^2 - v^2 + r^2)) /
9     // vector from ray origin to center
10     Vec3 v = m_center - ray.origin();
11     float sp = v * ray.direction();
12     float discriminant = sp*sp - v*v + m_radius * m_radius;
13     if (discriminant < 0) return false;
14     float root = sqrt(discriminant);
15     t = sp - root;
16     if (t < 0) t = sp + root;
17     if (t < 0) return false;
18     return true;
19 }
```

Sphere opp

EBNF: Burger

EBNF - Master Solution

```
1 // Consumes next Bun = "B".
2 bool Bun(std::istream& is) {
3     return consume(is, 'B'); // Already implemented for you
4 }
5
6 // Consumes next Patties = "P" "P" .
7 bool Patties(std::istream& is) {
8     if (consume(is, 'P')) {
9         while (lookahead(is) == 'P' && consume(is, 'P')) {
10             return true;
11         }
12         return false;
13     }
14 }
15
16 // Consumes next Onions = "O" "O" "O" { Onions }.
17 bool Onions(std::istream& is) {
18     if (consume(is, 'O')) {
19         unsigned int count = 1;
20         while (lookahead(is) == 'O' && consume(is, 'O')) {
21             ++count;
22         }
23         return count % 3 == 0;
24     }
25     return false;
26 }
27
28 // Consumes next Hamburger = Bun { Onions } Patties Bun.
29 bool Hamburger(std::istream& is) {
30     if (Bun(is)) {
31         if (lookahead(is) == 'O' && !Onions(is)) {
32             return false;
33         }
34         if (!Patties(is) && Bun(is)) {
35             return false;
36         }
37         return !lookahead(is);
38     }
39     return false;
40 }
```

> Console

Tree - Master Solution

```
1 // POST: Delete a leaf node with given value from the (sub)tree starting at root.
2 // If a leaf node is deleted, remove it from its parent.
3 // If an inner node becomes empty, remove it from its parent and delete it.
4 // Return true if and only if the (sub)tree starting at root was deleted.
5 bool deleteLeaf(Node* root, unsigned int value) {
6     if (root == nullptr) {
7         return false;
8     }
9     if (root->isLeaf() && root->value == value) {
10         delete root;
11         return true;
12     }
13     for (unsigned int i = 0; i < root->children.size(); ++i) {
14         if (deleteLeaf(root->children[i], value)) {
15             root->removeFromChildren(root->children[i]);
16         }
17         if (root->children.size() == 0) {
18             delete root;
19             return true;
20         }
21     }
22     return false;
23 }
```

Task 2: Dynamic Queue - Master Solution

```
1 #include <iostream>
2 #include <cassert>
3 #include "Queue.h"
4
5 // Queue::Queue() : first(nullptr), last(nullptr) {}
6
7 void Queue::is_valid() const {
8     assert((first == nullptr) == (last == nullptr));
9 }
10
11 void Queue::enqueue(int value) {
12     is_valid();
13     Node* node = new Node(value, nullptr);
14     if (first == nullptr) {
15         first = node;
16         last = node;
17     } else {
18         last->next = node;
19         last = node;
20     }
21 }
22
23 int Queue::dequeue() {
24     is_valid();
25     assert(!is_empty());
26     Node* node = first;
27     first = first->next;
28     if (first == nullptr) {
29         last = nullptr;
30     }
31     int value = node->value;
32     // A proper solution should also free the memory used by the first node.
33     // In the upcoming week, we will learn the operator delete that can be
34     // used for this purpose:
35     delete node;
36     return value;
37 }
38
39 bool Queue::is_empty() const {
40     is_valid();
41     return first == nullptr;
42 }
43
44 // PRE: -
45 // POST: print the sequence of integers in nodes reachable from n,
46 // in reverse order, with one space after each integer.
47 void print_node_reverse(std::ostream& os, Node* n) {
48     if (n != nullptr) {
49         print_node_reverse(os, n->next);
50         os << n->value << " ";
51     }
52 }
53
54 void Queue::print_reverse(std::ostream& os) const {
55     os << "[";
56     if (first != nullptr) {
57         print_node_reverse(os, first->next);
58         os << first->value;
59     }
60     os << "]";
61 }
62
63 // PRE: -
64 // POST: print the sequence of integers in nodes reachable from n,
65 // in order, with one space before each integer.
66 void print_node(std::ostream& os, Node* n) {
67     if (n != nullptr) {
68         os << n->value;
69         print_node(os, n->next);
70     }
71 }
72
73 void Queue::print(std::ostream& os) const {
74     os << "[";
75     if (first != nullptr) {
76         os << first->value;
77         print_node(os, first->next);
78     }
79     os << "]";
80 }
```

> Console

EBNF: Train

Train (EBNF) - Master Solution

```
1 #include <iostream>
2 #include "ebnf_helpers.h"
3
4 // PRE: Valid input stream is.
5 // POST: Returns true if one sequence of (1 to 3) Passengers cars,
6 // followed by a Diner car, could be parsed successfully; false otherwise.
7 bool MiddleCars(std::istream& is) {
8     // BEGIN OF SOLUTION
9     unsigned int count = 0;
10    while (lookahead(is) == 'P') {
11        if (!consume(is, "P")) return false;
12        ++count;
13    }
14    if (count > 0 && count <= 3) {
15        return consume(is, "D");
16    }
17    return false;
18    // END OF SOLUTION
19 }
20
21 // PRE: Valid input stream is.
22 // POST: Returns true if a valid train sequence could be parse successfully;
23 // false otherwise.
24 bool Train(std::istream& is) {
25     if (consume(is, "E")) {
26         // BEGINNING OF SOLUTION
27         while (lookahead(is) == 'B') {
28             if (!RowCars(is)) return false;
29         }
30         while (lookahead(is) == 'P') {
31             if (!MiddleCars(is)) return false;
32         }
33         if (!ServiceCars(is)) return false;
34     }
35     // After consuming ServiceCars, make sure we have reached the end of the string
36     return lookahead(is) == '#';
37     // END OF SOLUTION
38 }
39
40 }
41 return false;
42 }
```

EBNF: Princess

→ Le prime funzioni usano solo consume.

```
// Frogs = Frog Frog {Frog Frog};
bool Frogs(std::istream& is) {
    unsigned int counter = 0;
    if (Frog(is)) {
        ++counter;
        while (Frog(is)) {
            ++counter;
        }
        if (counter == 0) {
            return false;
        }
        return (counter%2 == 0);
    }
}

// Peas = Pea { Pea } On Ground.
bool Peas(std::istream& is) {
    bool valid = false;
    if (Pea(is)) {
        valid = true;
        while (Pea(is)) {
            valid = valid && true;
        }
        return valid && On(is) && Ground(is);
    }
    return false;
}

// Stack = Mattress { Frogs | Stack } Mattress.
bool Stack(std::istream& is) {
    if (Mattress(is) || Stack(is) && Mattress(is)) {
        return true;
    }
    return false;
}

// Night = Princess On Pile On Peas.
bool Night(std::istream& is) {
    return Princess(is) && On(is) && Stack(is) && On(is) && Peas(is);
}
```

Character rotation (Q12) - Master Solution

```
1 #include <cassert>
2
3 // PRE: Input capital character between A and Z
4 // POST: Returns the input character rotated by key (recall Caesar's encoding)
5 char rot(char input, unsigned int key) {
6     assert(input >= 65 && input <= 90);
7
8     return (input - 65 + key) % 26 + 65;
9 }
10
```

Binary Tree - Student Attempt

```
1 #ifndef BINARY_TREE_HELPER_H
2 #define BINARY_TREE_HELPER_H
3
4 #include "Node.h"
5
6 // pre: proper (sub-)tree starting from node root (i.e., in particular no cycles)
7 // post: return the number of full nodes contained in the (sub-)tree starting at root
8 unsigned int nonFullNodes(Node& root) {
9     unsigned int counter = 0;
10    if (root == nullptr) {
11        return 0;
12    }
13    if (root->left == nullptr && root->right != nullptr) {
14        counter += 1 + nonFullNodes(root->right);
15    }
16    if (root->left != nullptr && root->right == nullptr) {
17        counter += 1 + nonFullNodes(root->left);
18    }
19    if (root->left != nullptr && root->right != nullptr) {
20        counter += nonFullNodes(root->left) + nonFullNodes(root->right);
21    }
22    return counter;
23 }
24 }
```

```
// pre: proper (sub-)tree starting from node root (i.e., in particular no cycles)
// called "original tree"
// post: maximum (sub-)tree starting from root contained in the original tree
// such that all nodes are full.
void prune(Node& root) {
    if (root->left != nullptr || root->right != nullptr) {
        Node* p = root->right;
        root->right = nullptr;
        delete p;
    } else if (root->right == nullptr) {
        Node* p = root->left;
        root->left = nullptr;
        delete p;
    } else {
        prune(root->left);
        prune(root->right);
    }
}

#endif
```

Transitive Closure - Master Solution

```
1 #include "equality.h"
2
3 using Symbol = char;
4 using Equalities = std::set<Equality>;
5
6 // POST: Returns true if this equality shares a symbol with the other equality.
7 // E.g. "x = y" and "y = z" share symbol y.
8 // If a symbol is shared, first and second will be set to the remaining
9 // two symbols. Otherwise, first and second will be set to 0.
10 // Examples (inputs -> return value / first / second):
11 // "a = y", "y = z" -> true / x, z
12 // "a = b", "a = c" -> true / b, c
13 // "x = y", "x = y" -> true / y, y
14 // "p = q", "a = b" -> false / 0, 0
15 bool Equality::overlaps(const Equality& other, Symbol& first, Symbol& second) const {
16     first = 0;
17     second = 0;
18
19     if (_left == other._left) {
20         first = _right;
21         second = other._right;
22     } else if (_left == other._right) {
23         first = _right;
24         second = other._left;
25     } else if (_right == other._left) {
26         first = _left;
27         second = other._right;
28     } else if (_right == other._right) {
29         first = _left;
30         second = other._left;
31     }
32
33     return first != 0 && second != 0;
34 }
```

Transitive Closure - Master Solution

```
1 // POST: Transitivity closes the given set of equalities.
2 // E.g. the set {x=y, y=z} is changed to {x=y, y=z, x=z}.
3 // HINT: You can ignore trivial (reflexive) equivalences, e.g. x=x.
4 // The tests/autograder ignores them as well, so it doesn't matter
5 // whether or not you compute them.
6 void transitively_close(Equality& equalities) {
7     Equalities newly_inferred;
8
9     do {
10         newly_inferred.clear();
11
12         // NOTE: As far as I could figure out (cppreference.com), starting with
13         // C++17, it is safe to insert into a std::set while iterating over it.
14         // However, this suggests that it may not be safe prior to C++17. Hence,
15         // I insert into a dedicated (initially empty) set, and merge the two
16         // later on.
17
18         for (const Equality& eq1 : equalities) {
19             for (const Equality& eq2 : equalities) {
20                 Symbol p, q;
21
22                 if (eq1.overlaps(eq2, p, q)) {
23                     Equality eq3 = Equality(p, q);
24
25                     if (equalities.find(eq3) == equalities.end()) {
26                         newly_inferred.insert(eq3);
27                     }
28                 }
29             }
30         }
31
32         equalities.insert(newly_inferred.begin(), newly_inferred.end());
33     } while (!newly_inferred.empty());
34 }
```

Second Smallest (Q14) - Master Solution

```
1 #include <iostream>
2 #include <vector>
3
4 // PRE: a reference to a valid std::vector 'v'
5 // The vector must contain unique values (no value may appear more than once)
6 // POST: returns a pointer to the second-smallest element of the vector.
7 // If the vector contains less than two elements, returns nullptr.
8 // Example: if v is {4, 1, 2, 3}, the function returns a pointer to "2"
9 int* return_second_smallest(std::vector<int>& v) {
10     // BEGIN SOLUTION
11     if (v.size() < 2)
12         return nullptr;
13
14     int* smallest = nullptr;
15     int* second_smallest = nullptr;
16     for (unsigned int i = 0; i < v.size(); ++i) {
17         if (smallest == nullptr || v[i] < *smallest) {
18             second_smallest = smallest;
19             smallest = &v[i];
20         } else if (second_smallest == nullptr || v[i] < *second_smallest) {
21             second_smallest = &v[i];
22         }
23     }
24
25     return second_smallest;
26     // END SOLUTION
27 }
```

```
bool isPalindromeRecursive(std::string::const_iterator begin,
                           std::string::const_iterator end) {
    if (begin == end) {
        return true;
    }
    if (begin == (end-1)) {
        return true;
    }
    if (*begin != *(end-1)) {
        return false;
    }
    return isPalindromeRecursive(++begin, --end);
}
```

Q16 - Master Solution Rotating Trees (Q17) - Student Answer Project file system Project info	Q16 - Master Solution Rotating Trees (Q17) - Student Answer
<pre> 1 #include "knapsack.h" 2 3 // PRE: begin and end iterators must originate from a valid vector (may be empty) 4 // POST: Maximum value of filled knapsack 5 int knapsack(unsigned int max_weight, std::vector<item>::iterator begin, std::vector<item>::iterator end, unsigned int depth) { 6 // You can (but don't have to) use depth to identify and to help debug 7 // the recursion. For that, pass depth + 1 to each recursive call. 8 std::string indentation = std::string(2 * depth, " "); 9 std::cout << indentation << "max_weight = " << max_weight << "\n"; 10 11 if (max_weight == 0 begin == end) { 12 std::cout << indentation << "no weight or no items left!\n"; 13 return 0; 14 } 15 16 item cur = *begin; 17 begin++; 18 if (cur.weight > max_weight) { 19 // std::cout << indentation << "item is too heavy!\n"; 20 return knapsack(max_weight, begin, end, depth + 1); 21 } else { 22 // Two cases 23 // Case 1: we choose to take the item 24 std::cout << indentation << "max_weight = " << max_weight << " choose to take the item, new weight = " << max_weight - cur.weight << "\n"; 25 int val1 = cur.value + knapsack(max_weight - cur.weight, begin, end, depth + 1); 26 std::cout << indentation << "resulting value = " << val1 << "\n"; 27 // Case 2: we choose not to take the item 28 std::cout << indentation << "max_weight = " << max_weight << " choose NOT to take the item!\n"; 29 std::cout << indentation << "resulting value = " << val2 << "\n"; 30 if (val1 > val2) 31 return val1; 32 else 33 return val2; 34 } 35 } </pre>	<pre> 1 #include "node.h" 2 3 // PRE: subroot not null 4 // POST: subtree with root subtree memory leak free deleted 5 void delete_subtree(Node* subroot) { 6 if (subroot != nullptr) { 7 if (subroot->left != nullptr) { 8 delete_subtree(subroot->left); 9 subroot->left = nullptr; 10 } 11 if (subroot->right != nullptr) { 12 delete_subtree(subroot->right); 13 subroot->right = nullptr; 14 } 15 delete subroot; 16 } 17 return; 18 } 19 20 // PRE: subroot and subroot->left not null 21 // POST: subtree after being rotated to the right, i.e. the new root node 22 Node* right_rotate(Node* subroot) { 23 Node* res = subroot->left; 24 Node* res_left = res->right; 25 res->right = subroot; 26 subroot->left = res_left; 27 return res; 28 } 29 </pre>

Random functions

Q16 - Master Solution Rotating Trees (Q17) - Student Answer	Q16 - Master Solution Rotating Trees (Q17) - Student Answer
<pre> // PRE: Iterator pair [begin,end) specifies a valid (possibly empty) range // within a vector. // POST: Returns the length of the sorted prefix starting from begin. // Sorted means that the elements appear in non-decreasing // order. The empty range is assumed to have a sorted prefix // of length 0. unsigned int length_sorted_prefix(iterator begin, iterator end) { if (begin == end) return 0; // Iterate over the while range, // and count subsequent non-decreasing numbers unsigned int c = 1; for (i; begin != end - 1 && *begin <= *(begin+1); ++begin) { ++c; } return c; } // pre: non-negative numbers left, right, but not both left == right == 0 // post: return greatest common divisor of left and right unsigned int gcd(unsigned int left, unsigned int right) { while (left != 0) { unsigned int temp = right % left; right = left; left = temp; } return right; } </pre>	<pre> // pre: non-negative numbers left and right // post: return the least common multiple (LCM) of left and right // if left == 0 or right == 0 then 0 // otherwise the LCM is the smallest number n // such that left and right both divide n // (i.e. n is a multiple of both left and right) unsigned int lcm(unsigned int left, unsigned int right) { // hint: if you happen know the formula, you are allowed to use the gcd function from above // if not, you can also implement a naive version if (left == 0 right == 0) { return 0; } else { const bool naive = false; if (naive) { return left * gcd(left, right) / right; } //overflow version: //return left * right / gcd(left, right); } else { // naive alternative unsigned int l = left; while (l % right != 0) l += left; return l; } } </pre>
<pre> 36 // PRE: n is strictly greater than 2. 37 // POST: Returns the largest prime number strictly smaller than n. 38 // Examples: 39 // * n == 5 -> 3 40 // * n == 10 -> 7 41 // * n == 100 -> 97 42 // * n == 1000 -> 997 43 // * n == 1000000 -> 999983 44 // * n == 1000000000 -> 999999983 45 // * n == 1000000000000 -> 999999999983 46 // * n == 10000000000000 -> 9999999999983 47 // * n == 100000000000000 -> 99999999999983 48 // * n == 1000000000000000 -> 999999999999983 49 // * n == 10000000000000000 -> 999999999999983 50 // * n == 100000000000000000 -> 999999999999983 51 // * n == 1000000000000000000 -> 999999999999983 52 // * n == 10000000000000000000 -> 999999999999983 53 // * n == 100000000000000000000 -> 999999999999983 54 // * n == 1000000000000000000000 -> 999999999999983 55 // * n == 10000000000000000000000 -> 999999999999983 56 // * n == 100000000000000000000000 -> 999999999999983 57 // * n == 1000000000000000000000000 -> 999999999999983 58 // * n == 10000000000000000000000000 -> 999999999999983 59 // * n == 100000000000000000000000000 -> 999999999999983 60 // * n == 1000000000000000000000000000 -> 999999999999983 61 // * n == 10000000000000000000000000000 -> 999999999999983 62 // * n == 100000000000000000000000000000 -> 999999999999983 63 // * n == 1000000000000000000000000000000 -> 999999999999983 64 // * n == 10000000000000000000000000000000 -> 999999999999983 </pre>	<pre> 67 // PRE: Two iterator pairs [first_begin, first_end), and [second_begin, 68 // second_end) specify two valid ranges within two distinct vectors. 69 // POST: Returns an iterator to the starting position of the first occurrence, 70 // of the values from the second range, in the first range. 71 // You can assume that the second range is shorter than the first range, 72 // and that the second range occurs at least once in the first range. 73 // NOTE: the use of the function std::find(), or a similar library 74 // function, is prohibited. 75 // Examples: 76 // * [0,1,2,0,1,2], [0,1] -> iterator to left-most 0 in first range 77 // * [0,1,2,0,1,2], [1,2] -> iterator to the right-most 1 in the first range 78 // * [0,1,2,0,1,2], [1,2] -> iterator to the right-most 1 in the first range 79 // * [0,1,2,0,1,2], [1,2] -> iterator to the right-most 1 in the first range 80 // * [0,1,2,0,1,2], [1,2] -> iterator to the right-most 1 in the first range 81 // * [0,1,2,0,1,2], [1,2] -> iterator to the right-most 1 in the first range 82 // * [0,1,2,0,1,2], [1,2] -> iterator to the right-most 1 in the first range 83 // * [0,1,2,0,1,2], [1,2] -> iterator to the right-most 1 in the first range 84 // * [0,1,2,0,1,2], [1,2] -> iterator to the right-most 1 in the first range 85 // * [0,1,2,0,1,2], [1,2] -> iterator to the right-most 1 in the first range 86 // * [0,1,2,0,1,2], [1,2] -> iterator to the right-most 1 in the first range 87 // * [0,1,2,0,1,2], [1,2] -> iterator to the right-most 1 in the first range 88 // * [0,1,2,0,1,2], [1,2] -> iterator to the right-most 1 in the first range 89 // * [0,1,2,0,1,2], [1,2] -> iterator to the right-most 1 in the first range 90 // * [0,1,2,0,1,2], [1,2] -> iterator to the right-most 1 in the first range 91 // * [0,1,2,0,1,2], [1,2] -> iterator to the right-most 1 in the first range 92 // * [0,1,2,0,1,2], [1,2] -> iterator to the right-most 1 in the first range 93 // * [0,1,2,0,1,2], [1,2] -> iterator to the right-most 1 in the first range 94 // * [0,1,2,0,1,2], [1,2] -> iterator to the right-most 1 in the first range 95 // * [0,1,2,0,1,2], [1,2] -> iterator to the right-most 1 in the first range 96 // * [0,1,2,0,1,2], [1,2] -> iterator to the right-most 1 in the first range 97 // * [0,1,2,0,1,2], [1,2] -> iterator to the right-most 1 in the first range 98 // * [0,1,2,0,1,2], [1,2] -> iterator to the right-most 1 in the first range 99 // * [0,1,2,0,1,2], [1,2] -> iterator to the right-most 1 in the first range 100 // * [0,1,2,0,1,2], [1,2] -> iterator to the right-most 1 in the first range </pre>

Q15 Priority Queue - Master Solution

```

1 #include <cassert>
2 #include "PriorityQueue.hpp"
3
4 // POST: Returns true if this node goes before the other node, according to
5 // the customer queueing order described in the exercise.
6 bool Node::before(Node& other) const {
7     return
8         weight > other.weight ||
9         (weight == other.weight && ticket_no < other.ticket_no);
10 }
11
12 // POST: Deconstructs the whole queue, which includes deallocating
13 // all its nodes
14 PriorityQueue::~PriorityQueue() {
15     while (head != nullptr) {
16         Node* tmp = head;
17         head = head->next;
18         delete tmp;
19     }
20 }
21
22 // PRE: Queue is not empty
23 // POST: Returns the number of the ticket that is first in the queue,
24 // removes the corresponding node, and frees the node's memory
25 unsigned int PriorityQueue::pop() {
26     Node* match = head;
27     unsigned int ticket_no = match->ticket_no;
28
29     head = head->next;
30     delete match;
31
32     return ticket_no;
33 }

```

```

35 // PRE: _weight != 0, and each ticket numbers is unique
36 // PRE: _ticket_no != 0, and each ticket numbers is unique
37 // POST: Inserts a new node, with the given weight and ticket number,
38 // at its right place in the queue, i.e. according to the queueing policy
39 void PriorityQueue::insert(unsigned int _weight, unsigned int _ticket_no) {
40     Node* node = new Node(_weight, _ticket_no);
41
42     if (empty()) {
43         // Case 1: queue is empty.
44         head = node;
45     } else if (node->before(*head)) {
46         // Case 2: non-empty queue, new node becomes new head.
47         node->next = head;
48         head = node;
49     } else {
50         // Case 3: non-empty queue, new node goes further back into the queue.
51         // Traverse queue, such that pred eventually points to the new node's
52         // new predecessor node.
53         Node* pred = head;
54         while (pred->next != nullptr && pred->next->before(*node)) {
55             pred = pred->next;
56         }
57
58         node->next = pred->next;
59         pred->next = node;
60     }
61 }

```

Strings, Numbers, and Recursion - Master Solution

```

6 bool isValid(string num) {
7     if (num.size() > 1 && num[0] == '0')
8         return false;
9     return true;
10 }
11
12 // PRE: A string of digits and a position
13 // POST: Returns the integer value at pos of the string a. If pos is out of bound then 0 must be returned.
14 int val(string a, unsigned int pos) {
15     if (pos >= a.length())
16         return 0;
17     return (a[pos] - '0');
18 }
19
20
21 // PRE: Three strings of digits a, b and c
22 // POST: True if and only if there exists an additive sequence, otherwise false
23 // res should hold the elements of the additive sequence
24 // e.g. res = ["100", "199", "299"] for a = "1", b = "99" and c = "100199299"
25 bool checkAddition(vector<string>& res, string a, string b, string c) {
26     if (!isValid(a) || !isValid(b))
27         return false;
28     string sum = addString(a, b);
29
30     if (sum == c) {
31         res.push_back(sum);
32         return true;
33     }
34
35     if (c.size() <= sum.size() || sum != c.substr(0, sum.size()))
36         return false;
37     else {
38         res.push_back(sum);
39
40         return checkAddition(res, b, sum, c.substr(sum.size()));
41     }
42 }

```

```

44 // PRE: A string of digits
45 // POST: Additive sequence of num as a vector if one exists, otherwise an empty vector should be returned.
46 vector<string> additiveSequence(string num) {
47     vector<string> res;
48     int l = num.length();
49
50     for (int i = 1; i <= l/2; i++) {
51         for (int j = 1; j <= (l - i)/2; j++) {
52             if (checkAddition(res, num.substr(0, i), num.substr(i, j), num.substr(i + j))) {
53                 res.insert(res.begin(), 1, num.substr(i, j));
54                 res.insert(res.begin(), 1, num.substr(0, i));
55                 return res;
56             }
57         }
58     }
59     res.clear();
60     return res;
61 }

```

Recursion: Rod Cutting - Master Solution

```

1 #include <vector>
2 #include <iostream>
3
4 using prices = std::vector<unsigned int>;
5
6 // pre: p.size() >= length + 1
7 // post: return the best value that can be attained by
8 // cutting a rod of the given length and given
9 // prices per length
10 unsigned int bestValue(const prices& p, unsigned int length) {
11
12     if (length <= 0) return 0;
13
14     unsigned int bv = 0;
15     for (unsigned int i = 1; i < p.size(); ++i) {
16         if (i <= length) {
17             unsigned int nv = p.at(i) + bestValue(p, length - i);
18             if (bv < nv) bv = nv;
19         }
20     }
21
22     return bv;
23 }

```